

ADR 0009 — Core, CLI and web workspace boundaries

Date: 2026-09-12 · Status: proposed · Milestone: v2.0 · Issue: #28 (children #88, #89)

Context

BackupSage has one Cargo package. Its reusable indexing pipeline constructs terminal progress bars, prints source headers and warnings, and its index discovery prints a hint. A second frontend would inherit these side effects and the CLI dependencies. The public JSON fixtures, binary name, arguments and exit codes already form a compatibility contract (ADR 0003).

This is two independently testable changes: first separate reporting and cancellation from rendering (#88), then extract the workspace members (#89). The PRs land in that order. The empty web member is part of the extraction; it has no independent behavior to justify a third implementation issue.

Decision

- The root `backupsage` package remains the CLI, with the existing binary, command parser, terminal tables, sanitization and progress renderer. Its library re-exports core modules so existing `backupsage::` callers and integration tests retain their imports.
- `backupsage-core` owns indexing, format detection, content hashing and media metadata, directory walking, SQLite stores, master catalog, search, deduplication, report/plan data, output

safety and the existing sealed Borg read boundary. It depends on neither frontend and contains no terminal printing, ANSI rendering, clap, indicatif, comfy-table or Axum dependency.

- `backupsage-web` is a library placeholder depending only on core. It has no server, route, transport type, executable or web framework. Issues #29, #30 and #31 remain separate work. Core has no knowledge of this member.
- Member directories are `src/core` and `src/web`, keeping this extraction inside the authorized paths. Cargo's workspace dependency graph, rather than a conventional `crates/` directory name, enforces ownership. All three members are default members so existing root CI commands also run core unit tests. The root retains integration tests and fixtures.
- `OperationControl` accepts a synchronous `ProgressObserver` and a clonable `CancellationToken`. Events distinguish source/destination, byte/file totals, advances, entries, warnings, discovery, readiness to promote and successful completion. Event data is not terminal-sanitized; frontends escape it for their own output context. Warning text remains unchanged and is not a new serialized public contract.
- Existing library entry points delegate to a silent, non-cancelling default control. The CLI explicitly supplies `TerminalProgress`, which owns the existing output text, sanitization, truncation and bar styles. This intentionally removes unsolicited output from library calls while preserving CLI behavior. No new command or cancellation flag is added.
- Cancellation returns the typed `Cancelled` error. Indexing checks at directory-walk steps, content-read chunks, compressed-input reads and before promotion. The staged-output guard still removes unfinished databases and preserves any previous completed index. An observer can cancel at `ReadyToPromote`; `Finished` means promotion succeeded. A cancellation arriving after the last check cannot undo promotion.

Alternatives considered

- **Move files without removing output.** Rejected: core would still pull terminal dependencies and impose presentation on the next frontend.
- **Feature-gate terminal code in core.** Rejected: feature unification could restore rendering dependencies; a separate crate boundary is directly inspectable and leaves core independent in every configuration.
- **Rename the CLI package and move all tests.** Deferred: retaining the root package preserves binary discovery, fixture paths, installation habits and source imports with a small compatibility facade.
- **Build a web server or introduce an async job framework now.** Deferred: there is no web behavior to implement in this issue. The typed observer can be adapted to that frontend without prescribing its runtime.

Consequences

- Dependencies point from CLI and web to core, with no reverse edge or cycle. Core-only builds can be checked independently of terminal code.
- Report and plan JSON, index schema, CLI arguments and exit codes do not change. Existing integration tests and golden fixtures remain unchanged; unit-test resource paths and documentation imports follow their owning crate when extracted. No dependency versions need to change.
- Cancellation is cooperative. It cannot interrupt a blocked OS read, decoder/image computation or SQLite operation while that call is running. It is not a timeout or a new Borg lifecycle mechanism. Borg's existing cancellation, capability gates and privileged runtime are unchanged.

- Core still uses filesystem, SQLite, compression and image libraries: frontend independence does not mean a computation-only or no-I/O crate.
- The compatibility facade preserves module names without duplicating implementation. Frontends should use core directly for new shared work.

Safety invariant

BackupSage never rewrites or deletes from an archive. This extraction does not relax output-path protection, staged promotion, Borg source refusal, or the sealed Borg runtime boundary. Cancelled indexing leaves the previous completed destination intact and does not publish a partial replacement.

— codex